DATA STRUCTURES · DIGITAL NOTES

UNIT V

Trees and Hashing

Trees · Binary Search Tree (insert, delete, traverse) · Hash functions · Collision handling · Hash tables

Roadmap of this unit

5.1	Introduction to trees & terminology
5.2	Binary trees and Binary Search Tree (BST)
5.3	BST insertion, deletion and traversal
5.4	Hashing: hash functions & hash tables
5.5	Collision resolution: chaining & open addressing
5.6	Applications of hashing: unique IDs, caching

5.1 Introduction to Trees

DEFINITION

Tree = a **non-linear, hierarchical** data structure made of **nodes** connected by **edges**, with one special node called the **root**. A tree with n nodes has exactly **n − 1 edges** and **no cycles**.

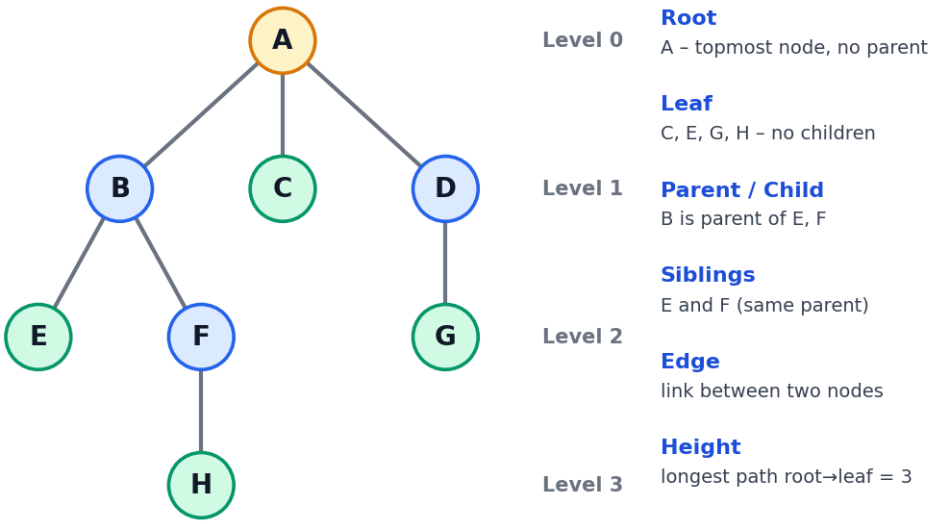


Fig: A general tree with its terminology

Term	Meaning
Root	Top node – the only node without a parent
Parent / Child	Node directly above / below another node
Siblings	Nodes with the same parent
Leaf (external node)	Node with no children
Internal node	Node with at least one child
Degree of a node	Number of children of that node

Term	Meaning
Level	Distance from the root (root = level 0)
Height of tree	Number of edges on the longest path from root to a leaf
Subtree	A node together with all its descendants
Ancestor / Descendant	Any node on the path up to the root / any node below

Why trees?

- Natural for **hierarchical data**: file systems, organisation charts, HTML DOM.
- Faster **search / insert / delete** than lists – O(log n) in a balanced tree.
- Basis of expression trees, heaps, tries, decision trees, database indexes.

5.2 Binary Trees and Binary Search Tree

DEFINITION
Binary tree = a tree in which every node has **at most two children**, called the **left child** and the **right child**.

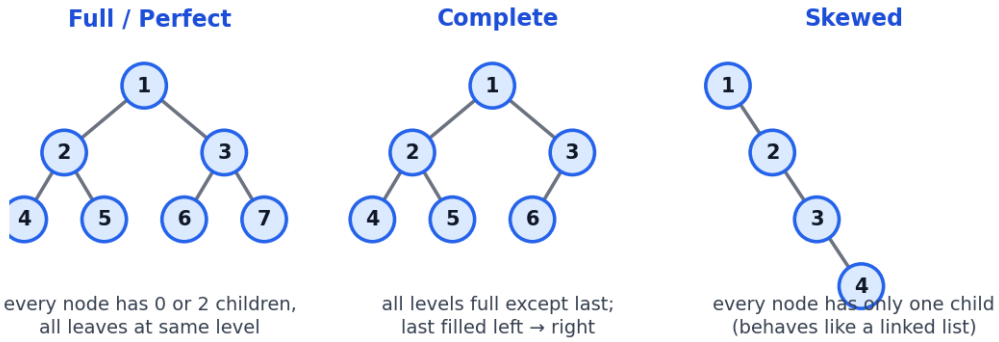


Fig: Common shapes of binary trees

- Maximum nodes at level L = 2^L.
- Maximum nodes in a tree of height h = $2^{h+1} - 1$.
- A binary tree with n nodes has height between $\log_2 n$ (balanced) and $n - 1$ (skewed).

```
struct Node {
    int data;
    struct Node *left, *right;          // two child pointers
};
struct Node* newNode(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));
    n->data = x;  n->left = n->right = NULL;
    return n;
}
```

Node of a binary tree

Binary Search Tree (BST)

DEFINITION
BST = a binary tree in which, for **every node**: all values in the **left subtree are smaller** and all values in the **right subtree are larger** than the node. (Duplicates are normally not allowed.)

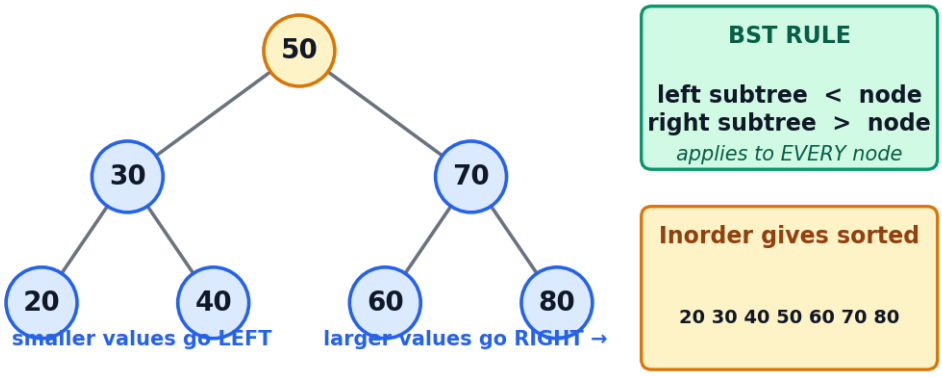


Fig: A BST – the inorder traversal always prints the keys in sorted order

Operation	Balanced BST	Skewed BST (worst)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Space	O(n)	O(n)

REMEMBER
Inserting already-sorted keys (10, 20, 30, 40…) creates a **skewed tree** that behaves like a linked list – this is why balanced trees (AVL, Red-Black) exist.

5.3 BST Operations

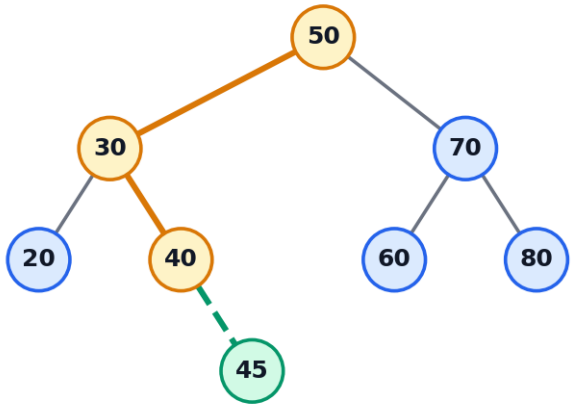
A. Searching

1. Start at the root.
2. If key == node → **found**.
3. If key < node → go **left**; if key > node → go **right**.
4. If you reach NULL → **not found**.

```
struct Node* search(struct Node *root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}
```

B. Insertion

- Search for the key; the **empty position where the search ends** is where the new node goes.
- A new node is **always inserted as a leaf**.



INSERT 45

① 45 < 50 → go LEFT

② 45 > 30 → go RIGHT

③ 45 > 40 → go RIGHT

④ right of 40 is empty

⑤ → place 45 here (as a leaf)

Fig: Inserting 45 – follow the comparison path and attach at the empty spot

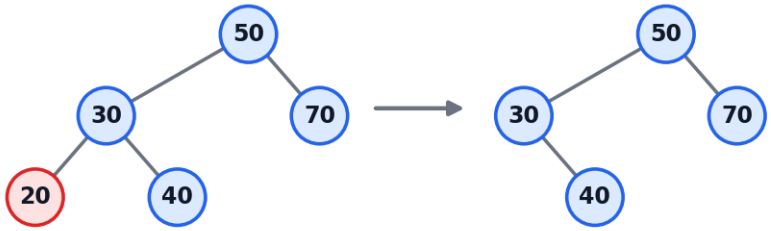
```
struct Node* insert(struct Node *root, int x) {
    if (root == NULL) return newNode(x);              // empty spot found
    if (x < root->data)      root->left  = insert(root->left, x);
    else if (x > root->data) root->right = insert(root->right, x);
    return root;                                      // duplicates ignored
}
```

Build a BST from the keys **50, 30, 70, 20, 40, 60, 80**: 50 becomes the root; 30 < 50 → left; 70 > 50 → right; 20 < 50, < 30 → left of 30; 40 → right of 30; 60 → left of 70; 80 → right of 70.

C. Deletion – three cases

Case	Node to delete has…	Action
1	**No child** (leaf)	Remove the node; set parent's pointer to NULL
2	**One child**	Replace the node by its child (parent links directly to the child)
3	**Two children**	Copy the **inorder successor** (smallest key in the right subtree) – or the inorder predecessor – into the node, then delete that successor (which has at most one child)

Case 1 – delete a LEAF node (20)



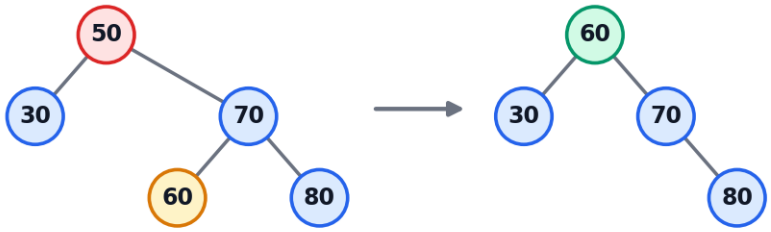
just remove it (parent's pointer = NULL)

Case 2 – delete a node with ONE child (30)



child takes the place of the deleted node

Case 3 – delete a node with TWO children (50)



replace with inorder successor 60 (smallest in right subtree), then delete 60

Fig: Deletion cases in a BST – red = node to delete, green = node that takes its place

```
struct Node* minNode(struct Node *t) { while (t->left) t = t->left; return t; }

struct Node* deleteNode(struct Node *root, int key) {
    if (root == NULL) return NULL;
    if (key < root->data)      root->left  = deleteNode(root->left, key);
    else if (key > root->data) root->right = deleteNode(root->right, key);
    else {                                              // node found
        if (root->left == NULL)  { struct Node *t = root->right; free(root); return t; }
        if (root->right == NULL) { struct Node *t = root->left;  free(root); return t; }  // 0/1 child
        struct Node *s = minNode(root->right);          // two children: inorder successor
        root->data = s->data;
        root->right = deleteNode(root->right, s->data);
    }
    return root;
}
```

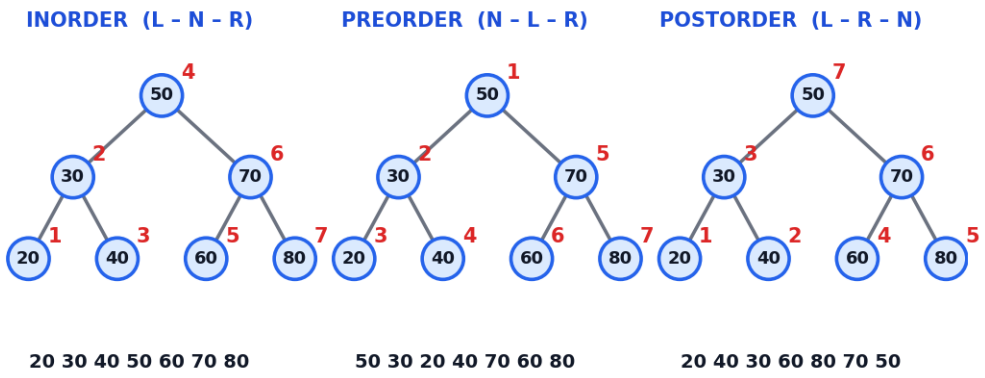
EXAM TIP

For the two-children case always write: "replace with **inorder successor** (leftmost node of the right subtree)", then show the diagram. This is a favourite 5-mark question.

5.4 Tree Traversals

Traversal = visiting every node exactly once. For a binary tree there are three depth-first orders (N = node, L = left, R = right) and one breadth-first order.

Traversal	Order	Result for the BST of Fig	Use
Inorder	L – N – R	20 30 40 50 60 70 80	Sorted output of a BST
Preorder	N – L – R	50 30 20 40 70 60 80	Copy a tree, prefix expression
Postorder	L – R – N	20 40 30 60 80 70 50	Delete a tree, postfix expression
Level order	level by level (queue)	50 30 70 20 40 60 80	BFS on a tree



red numbers = order in which nodes are visited

Fig: Traversal orders – red numbers show the visiting sequence

```
void inorder(struct Node *r) {
    if (r) { inorder(r->left);  printf("%d ", r->data);  inorder(r->right); }
}
void preorder(struct Node *r) {
    if (r) { printf("%d ", r->data);  preorder(r->left);  preorder(r->right); }
}
void postorder(struct Node *r) {
    if (r) { postorder(r->left);  postorder(r->right);  printf("%d ", r->data); }
}
```

Recursive traversals – only the position of printf changes

- **Level-order**: enqueue root; repeat – dequeue a node, print it, enqueue its left and right children (uses a **queue**, Unit IV).
- All traversals take **O(n)** time; recursive versions use O(h) stack space.
- **Memory trick:** the name tells where the **Node** is visited – **pre** = before children, **in** = between, **post** = after.

Reconstructing a tree: inorder + preorder (or inorder + postorder) together identify a unique binary tree. First element of preorder (or last of postorder) = root; elements left of the root in inorder form the left subtree.

5.5 Hashing

DEFINITION
Hashing = a technique that converts a **key** into an **array index** using a **hash function**, so that data can be stored and found in **O(1) average time**.

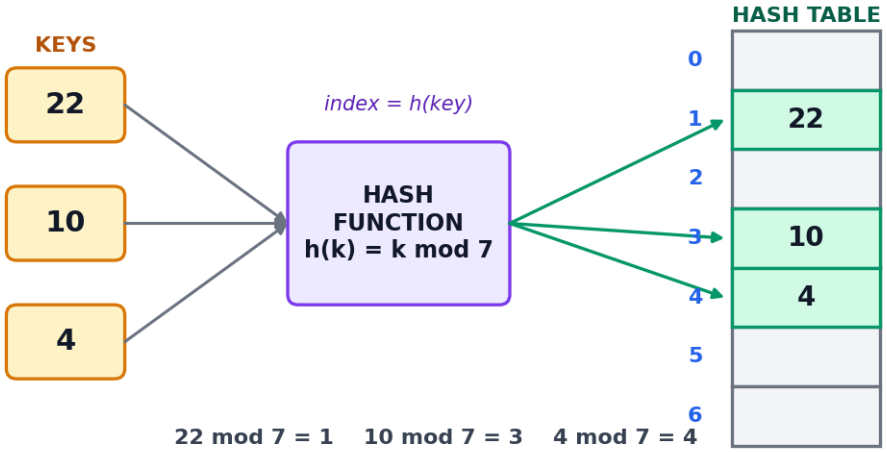


Fig: Keys are mapped to table slots by the hash function

Basic terms

Term	Meaning
Key	Value used to identify a record (roll no., username...)
Hash function h(k)	Formula that maps a key to an index 0 ... m−1
Hash table	Array of m slots (buckets) holding the records
Collision	Two different keys map to the **same** index
Load factor λ	λ = n / m (n = keys stored, m = table size) – measures how full the table is

Properties of a good hash function

- **Fast** to compute.
- **Uniform** – spreads keys evenly over the table (few collisions).
- **Deterministic** – same key always gives the same index.
- Uses **all** parts of the key.

Common hash functions

Method	Formula / idea	Example (m = 10 or 100)
Division	h(k) = k mod m (choose m as a **prime**)	1234 mod 7 = 2
Multiplication	h(k) = ⌊m · (k·A mod 1)⌋, 0 < A < 1 (A ≈ 0.618)	k = 123, m = 100 → 123 × 0.618 = 76.014 → frac 0.014 → h = 1
Mid-square	Square the key, take the middle digits	123^2 = 15129 → middle “51” → 51
Folding	Split the key into parts and add them	12345678 → 12 + 34 + 56 + 78 = 180 → 80 (last 2 digits)
String hashing	h = (Σ ascii(c_i) × 31^i) mod m	“AB” → (65×31 + 66) mod m

```
#define M 7
int hash(int key) { return key % M; }                // division method

int hashString(char *s) {                            // string hashing
    unsigned long h = 0;
    while (*s) h = h * 31 + *s++;
    return h % M;
}
```

Basic hash-table operations

Operation	Steps	Average	Worst
Insert(k)	compute h(k), place (resolve collision if any)	O(1)	O(n)
Search(k)	compute h(k), compare along the probe sequence / chain	O(1)	O(n)
Delete(k)	find the key, remove (mark deleted in open addressing)	O(1)	O(n)

5.6 Collision Resolution

A collision occurs when **$h(k_1) = h(k_2)$** for different keys. Two families of techniques handle it:

A. Separate chaining (closed addressing)

- Each slot holds a **linked list** (chain) of all keys that hash to it.
- Insert at the **head** of the list – O(1). Search = hash, then walk the list.
- Table never becomes “full”; load factor can exceed 1.
- Average search time ≈ **1 + λ**.
- Extra memory for pointers.

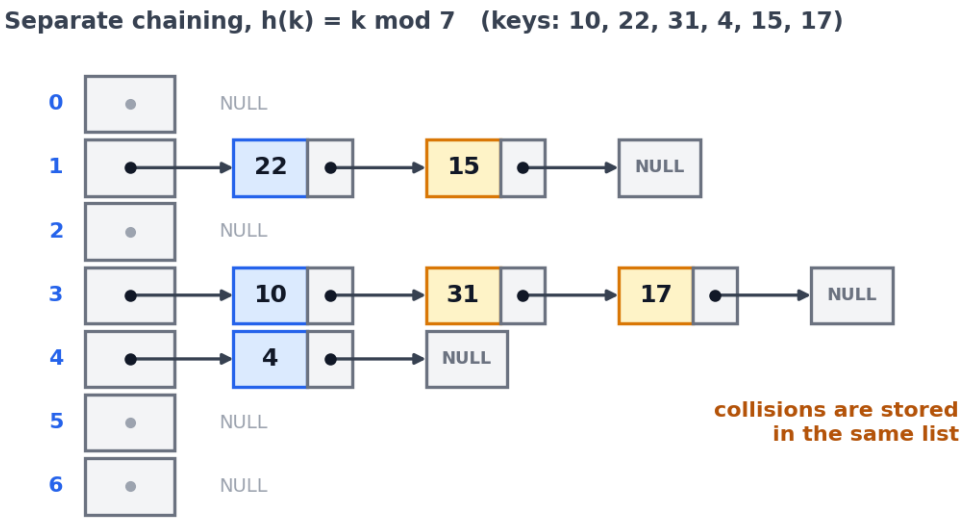


Fig: Chaining – keys 22 and 15 share slot 1; 10, 31 and 17 share slot 3

```
struct HNode { int key; struct HNode *next; };
struct HNode *table[M];                          // all NULL initially

void insertKey(int k) {
    int i = k % M;
    struct HNode *n = (struct HNode*) malloc(sizeof(struct HNode));
    n->key = k;  n->next = table[i];  table[i] = n;      // insert at head of chain
}
int searchKey(int k) {
    for (struct HNode *t = table[k % M]; t; t = t->next) if (t->key == k) return 1;
    return 0;
}
```

B. Open addressing (closed hashing)

All keys live **inside the table itself**. On collision we **probe** for another empty slot. Requires **λ ≤ 1**.

Technique	Probe sequence	Remark
Linear probing	(h(k) + i) mod m	Simple, but causes **primary clustering** (long runs of filled slots)
Quadratic probing	(h(k) + i²) mod m	Reduces primary clustering; may cause secondary clustering
Double hashing	(h₁(k) + i·h₂(k)) mod m	Best distribution; h₂(k) must never be 0, e.g. h₂ = 5 − (k mod 5)

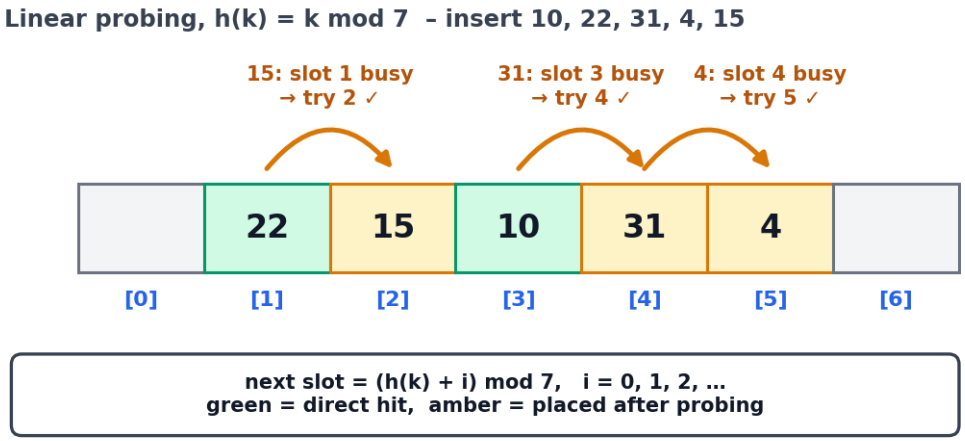


Fig: Linear probing – a colliding key moves to the next free slot

```
int table[M];                                   // -1 means empty
void insertLinear(int k) {
    int i = k % M, tries = 0;
    while (table[i] != -1 && tries++ < M) i = (i + 1) % M;   // probe next slot
    if (tries <= M) table[i] = k; else printf("Table full\n");
}
```

COMMON MISTAKE
In open addressing a deleted slot must be marked **“deleted” (tombstone)**, not simply emptied – otherwise searches for keys that probed past it will stop early and fail.

Basis	Chaining	Open addressing
Storage	Outside the table (linked lists)	Inside the table
Load factor	Can be > 1	Must be ≤ 1
Deletion	Easy	Needs tombstones
Clustering	None	Possible
Memory	Extra pointers	No pointers, better cache use

Trace (m = 7, linear probing): insert 10 → slot 3; 22 → 1; 31 → 3 busy → **4**; 4 → 4 busy → **5**; 15 → 1 busy → **2**. Final table: [_ , 22, 15, 10, 31, 4, _].

Rehashing: when λ becomes large (e.g. > 0.7) create a bigger table (about double, prime size) and re-insert every key – keeps operations near O(1).

5.7 Applications of Hashing

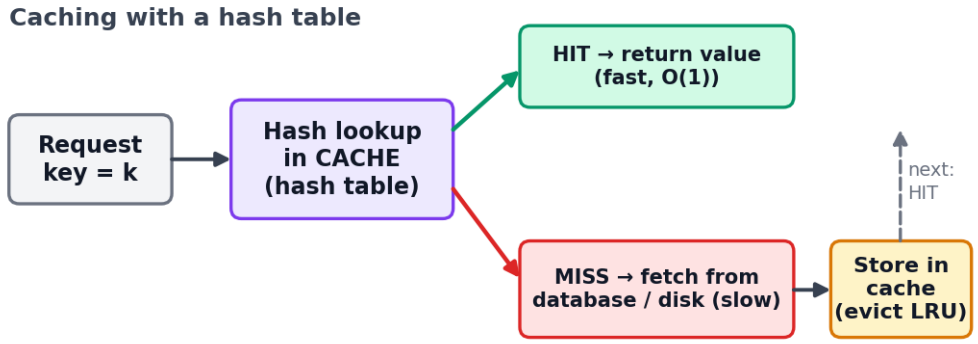


Fig: Caching – the hash table answers "is it already stored?" in O(1)

A. Unique identifier generation

- **Checksums / digests** – SHA-256 turns a file into a fixed-size fingerprint; equal fingerprints ⇒ same file (duplicate detection, integrity checks).
- **URL shorteners** – the long URL is hashed / encoded to a short unique code.
- **Version control** (Git) – every commit and file is identified by its hash.
- **Student / order IDs** – hash of name + timestamp gives an ID that is practically unique.
- **Blockchain** – each block carries the hash of the previous block.

B. Caching

- A **cache** stores recently used results so that they need not be recomputed or fetched again.
- A hash table gives **O(1) lookup** of cached entries – **hit** = found, **miss** = fetch and store.
- **LRU cache** = hash table (key → node) + doubly linked list (usage order). Both get and put are O(1); when full, the least-recently-used entry is evicted.
- Used in **browsers, CPU, DNS, databases, CDNs**.

C. Other applications

- **Dictionaries / symbol tables** in compilers and interpreters.
- **Database indexing** and joins.
- **Password storage** – only the hash of a password is stored.
- **Counting frequencies** (words, characters) and finding duplicates in O(n).
- **Sets and maps** (HashSet / HashMap, Python dict).

Structure	Search	Insert	Delete	Ordered?
Unsorted array / list	O(n)	O(1)	O(n)	No
Sorted array	O(log n)	O(n)	O(n)	Yes
BST (balanced)	O(log n)	O(log n)	O(log n)	Yes
Hash table	**O(1) avg**	O(1) avg	O(1) avg	No

5.8 Quick Revision & Practice

ONE-PAGE SUMMARY

- Tree: hierarchical, n nodes → n − 1 edges; binary tree ≤ 2 children.
- BST: left < node < right; inorder = sorted; search / insert / delete O(h).
- Delete: leaf → remove; one child → bypass; two children → inorder successor.
- Traversals: In = L N R, Pre = N L R, Post = L R N, Level = queue.
- Hashing: index = h(key); collision = two keys, same slot; load factor λ = n / m.
- Chaining = lists in slots; open addressing = probe (linear, quadratic, double).

Likely exam questions

2 marks

- Define binary search tree. Define root, leaf and height.
- What is a collision? What is load factor?
- Write the inorder traversal of a given BST.
- Differentiate chaining and open addressing.

5 marks

- Construct a BST from 45, 20, 60, 10, 30, 55, 70 and list all traversals.
- Explain deletion of a node with two children in a BST.
- Explain any three hash functions with examples.
- Insert 12, 18, 25, 32, 5 into a table of size 7 using linear probing.

10 marks

- Explain BST insertion, deletion and traversals with algorithms and diagrams.
- Explain hashing, collision resolution techniques and applications with examples.

Try it yourself

1. Insert **8, 3, 10, 1, 6, 14, 4, 7, 13** into an empty BST. Write the inorder, preorder and postorder sequences. *(Inorder: 1 3 4 6 7 8 10 13 14)*
2. Delete 3 from the tree above (it has two children). Which node replaces it? *(4 – inorder successor)*
3. Using h(k) = k mod 10 and chaining, store 31, 41, 51, 12, 22. Draw the table.
4. What is the height of a BST built from 1, 2, 3, 4, 5 inserted in this order? *(4 – skewed)*